coursera | UNIVERSITY OF MICHIGAN

Applied Information Extraction in Python

Today's goal • 3/3 items

Module 1
Information Extraction

Module 2
Named Entity Recognition (NER)

Named Entities and Named Entity Recognition

- What is Named Entity Recognition (NER)? Video • 5 min
- Knowledge Check: Named Entities and Named Entity Recognition Practice Assignment • Grade: 100%

Setting up NER as a Machine Learning Task

- BIO Encoding for Named Entity Labels Reading • 10 min
- BILOU Encoding for Named Entity Labels Reading • 10 min
- Machine Learning Fundamentals: How Machines Learn to Label Named Entities Reading • 10 min
- NER as a Sequence Classification Task Video • 8 min
- Knowledge Check: Setting up NER as a Machine Learning Task Practice Assignment • Grade: 100%

Hidden Markov Models (HMMs)

- Markov Chain and Hidden Markov Models Reading • 10 min
- Fundamentals of Markov Chain Models Video • 7 min
- Hidden Markov Models (HMMs) Video • 11 min
- **Training Hidden Markov Models: How HMMs Learn to Assign Labels** Reading • 10 min
- The Math Behind HMMs: How Probabilities Power Sequence Labeling Reading • 10 min
- Conditional Random Fields (CRFs) Video • 6 min
- Knowledge Check: Hidden Markov Models (HMMs) Practice Assignment • 15 min

How to Extract Information Using CRFs in Python?

- Demo: CRF Model Training Video • 8 min
- Jupyter Notebook Practice on Training CRFs Lab • 1h

Module Wrap-Up & Assignment

- Assignment 2 Introduction: Implementing a CRF Model Video • 4 min
- Build an Information Extraction Pipeline for CRF Based Extraction Programming Assignment • 3h
- Module 2 Assignment Graded Assignment • 30 min

Module 3
Neural Network Models

Module 4
Transformers Transform Information Extraction

Training Hidden Markov Models: How HMMs Learn to Assign Labels

0:45 / 4:34 2x

From Theory to Practice

Previously, we introduced Hidden Markov Models (HMMs), powerful probabilistic models that assume every word (observation) is generated by a hidden state (label). But knowing what an HMM is isn't enough. We also need to understand how to **train** it.

So what does "training" an HMM actually mean?

What Does It Mean to "Train" an HMM?

Training an HMM means estimating three sets of probabilities that define how the model behaves:

- **Initial probabilities**
  - What's the probability of starting in each hidden state (label)? E.g., how likely is a sentence to start with a PER tag?
- **Transition probabilities**
  - Given the current state, what's the probability of transitioning to the next one? E.g., how likely is it that a PER label is followed by another PER?
- **Emission probabilities**
  - Given a state, what's the probability it emits a particular word? E.g., how likely is the word "Obama" to be generated from the PER state?

These probabilities form the core parameters of the HMM. Training involves learning these parameters from data.

Training with Labeled vs. Unlabeled Data

There are two main scenarios for training HMMs: Supervised Training (Labeled Data), and Unsupervised Training (Unlabeled Data).

1. Supervised Training (Labeled Data)

If each word in your training data is already tagged (e.g., in BIO format), training is straightforward:

- Count how often transitions and emissions occur
- Normalize the counts to calculate probabilities

Example:

If you see PER → PER 60 times out of 100 total transitions from PER, then: **P(PER → PER) = 0.6**

This method is simple and efficient, but requires **fully labeled data**, which is often expensive and time-consuming to create.

2. Unsupervised Training (Unlabeled Data)

If you only have observed words (and no labels), you must **infer the most likely sequence of hidden states**.

This is where the **Expectation-Maximization (EM)** algorithm comes in, specifically, the **Baum-Welch algorithm**, which is a version of EM tailored for HMMs.

Note: The **Baum-Welch algorithm** is a method used to train Hidden Markov Models (HMMs) when the sequence labels are not provided. It's a specific version of the Expectation-Maximization (EM) algorithm that helps estimate transition and emission probabilities based on observed data.

How the Expectation-Maximization Algorithm Works

The EM algorithm works in two alternating steps:

- **Expectation (E-step):** Use the current guess of parameters to estimate the likely hidden state sequences for your data.
- **Maximization (M-step):** Recalculate the parameters (transition, emission, and initial probabilities) to maximize the likelihood of generating the observed data.

Then repeat the steps until the parameters converge (i.e., stop changing significantly).

You can think of it as: **"Guess the hidden labels → Re-learn the parameters → Re-guess the labels → Repeat."**

Why Training Matters

Accurate training is essential for:

- Making reliable predictions during tagging
- Reducing error in downstream models
- Building trust in how the model handles ambiguous words

Even a perfectly structured HMM won't perform well without well-trained parameters, it'll simply make poor guesses using poor probabilities.

What Are the Limitations?

While elegant, training HMMs has trade-offs:

- The EM algorithm can get stuck in **local optima**, it may not find the best parameters
- HMMs rely on **strong independence assumptions** (e.g., Markov property, conditional independence of observations)
- HMMs model the **joint distribution** of words and labels, even when we only care about the labels, adding computational overhead

These limitations are exactly why we move next to **Conditional Random Fields (CRFs)**, which address many of these issues.

Want to Go Deeper?

- **Speech and Language Processing (Jurafsky & Martin)**
  - Chapter 17: Sequence Labeling for Parts of Speech and Named Entities
- **Blog: "HMMs for Sequence Tagging" – by Michael Collins** Clear walkthrough of HMMs in the context of NLP

Reflection Prompt

Suppose you're given 1,000 clinical sentences with **no labels**:

- What challenges would you face when training an HMM from this data?
- How might **inaccurate estimates of emission probabilities** affect the model's output?

Assignment Preview

You'll apply these training principles soon when working through notebook exercises and comparing HMMs to CRFs.

Try to think about:

- What each parameter is really telling the model
- Where things might go wrong, and how that would affect performance

Go to next item ✓ Completed

Like Dislike Report an issue

Go to next item →